

# **OBJECT ORIENTED PROGRAMING**

## **OOP**

### **PRACTICALS**

**NAME:** SYEDA FAIZA ASLAM

**SECTION:** B

**ROLL NO:** CT-25064

**LAB NO:** 06

**COURSE CODE:** CT-260

**COURSE TEACHER:** Mr. AHMED ZAKI

## Exercise

1. Create a class named Shape. All our shapes will be inherited from this class. It will contain the following data members and functions:

numberOfSides, area,

parametrized constructor, accessors and mutators for the data members. Create

classes called Rectangle, Circle and Triangle, which are all inherited from the class

Shape. Create a class called Square which is inherited from Rectangle.

The derived

classes will have the following members:

Rectangle: length, width, parameterized constructor, generateArea( ) – should

place the result in area.

Circle: radius, parameterized constructor, generateArea( ) – should place the result in area

Triangle: height, base, parameterized constructor, generateArea( ) – should place

the result in area ( $\text{Area} = \text{height} * \text{base} / 2$ )

Square: It should have a parameterized constructor that takes one side as input. The constructor should call the constructor for the Rectangle class with that value

as parameters. checkSides( ); checks if both sides are equal. Sides are inherited from Rectangle. generateArea( ) – should place the result in area

You must make use of parameterized constructors to initialize the values.

## **SOURCE CODE:**

```
#include<iostream>
#include<cmath>
using namespace std;

class Shape
{
protected:
    int numberOfSides;
    double area;

public:
    Shape(int sides)
    {
        numberOfSides = sides;
        area = 0;
    }

    void setSides(int s)
    {
        numberOfSides = s;
    }

    int getSides()
    {
        return numberOfSides;
    }

    double getArea()
    {
        return area;
    }
};
```

```
class Rectangle : public Shape
{
protected:
    double length;
    double width;

public:
    Rectangle(double l, double w) : Shape(4)
    {
        length = l;
        width = w;
    }

    void generateArea()
    {
        area = length * width;
    }
};
```

```
class Circle : public Shape
{
private:
    double radius;

public:
    Circle(double r) : Shape(1)
    {
        radius = r;
    }

    void generateArea()
    {
        area = 3.1416 * radius * radius;
    }
};
```

```
};
```

```
class Triangle : public Shape
```

```
{
```

```
private:
```

```
    double height;
```

```
    double base;
```

```
public:
```

```
    Triangle(double h, double b) : Shape(3)
```

```
    {
```

```
        height = h;
```

```
        base = b;
```

```
    }
```

```
    void generateArea()
```

```
    {
```

```
        area = (height * base) / 2;
```

```
    }
```

```
};
```

```
class Square : public Rectangle
```

```
{
```

```
public:
```

```
    Square(double side) : Rectangle(side, side)
```

```
    {
```

```
    }
```

```
    void checkSides()
```

```
    {
```

```
        if(length == width)
```

```
            cout<<"Both sides are equal"<<endl;
```

```
        else
```

```
            cout<<"Sides are not equal"<<endl;
```

```
    }
```

```
void generateArea()
{
    area = length * width;
}
};

int main()
{
    Rectangle r(5,4);
    r.generateArea();
    cout<<"Rectangle Area: "<<r.getArea()<<endl;

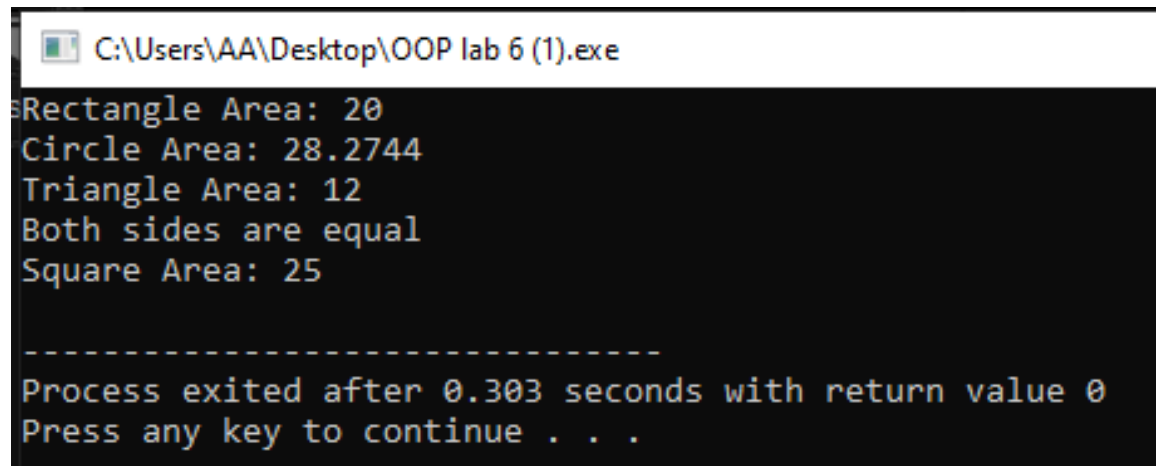
    Circle c(3);
    c.generateArea();
    cout<<"Circle Area: "<<c.getArea()<<endl;

    Triangle t(6,4);
    t.generateArea();
    cout<<"Triangle Area: "<<t.getArea()<<endl;

    Square s(5);
    s.checkSides();
    s.generateArea();
    cout<<"Square Area: "<<s.getArea()<<endl;

    return 0;
}
```

## OUTPUT:



```
C:\Users\AA\Desktop\OOP lab 6 (1).exe
Rectangle Area: 20
Circle Area: 28.2744
Triangle Area: 12
Both sides are equal
Square Area: 25

-----
Process exited after 0.303 seconds with return value 0
Press any key to continue . . .
```

2. Create a class called Calculator that has three private member variables Num1, Num2, Num3. In this class, you have to overload the functions for addition and multiplication such that they take two and three inputs respectively. You also have to make methods for subtraction and division. For example: add(1,2) and add(1,2,3) similarly for multiply, it would be multiply(1,2) and multiply(1,2,3). You may ask the user for input at the time of object creation. Afterwards just demonstrate how the functions are being called.

## SOURCE CODE:

```
#include<iostream>
using namespace std;

class Calculator
{
```

private:

```
double Num1, Num2, Num3;
```

public:

```
Calculator(double a, double b, double c=0)
```

```
{
```

```
    Num1 = a;
```

```
    Num2 = b;
```

```
    Num3 = c;
```

```
}
```

```
double add(double a, double b)
```

```
{
```

```
    return a + b;
```

```
}
```

```
double add(double a, double b, double c)
```

```
{
```

```
    return a + b + c;
```

```
}
```

```
double multiply(double a, double b)
```

```
{
```

```
    return a * b;
```

```
}
```

```
double multiply(double a, double b, double c)
```

```
{
```

```
    return a * b * c;
```

```
}
```

```
double subtract(double a, double b)
```

```
{
```

```
    return a - b;
```

```
}
```

```

double divide(double a, double b)
{
    if(b != 0)
        return a / b;
    else
    {
        cout<<"Division by zero not allowed"<<endl;
        return 0;
    }
}

};

int main()
{
    Calculator obj(10,5,2);

    cout<<"Add(10,5): "<<obj.add(10,5)<<endl;
    cout<<"Add(10,5,2): "<<obj.add(10,5,2)<<endl;

    cout<<"Multiply(10,5): "<<obj.multiply(10,5)<<endl;
    cout<<"Multiply(10,5,2): "<<obj.multiply(10,5,2)<<endl;

    cout<<"Subtract(10,5): "<<obj.subtract(10,5)<<endl;
    cout<<"Divide(10,5): "<<obj.divide(10,5)<<endl;

    return 0;
}

```

## OUTPUT:

```
C:\Users\AA\Desktop\OOP lab 6 (2).exe
Add(10,5): 15
Add(10,5,2): 17
Multiply(10,5): 50
Multiply(10,5,2): 100
Subtract(10,5): 5
Divide(10,5): 2

-----
Process exited after 0.3132 seconds with return value 0
Press any key to continue . . .
```

3. Create a class called Vector which represents a two-dimensional vector with x and y components. The class should have the following member functions:

- A constructor that initializes the x and y components of the vector.
- An overloaded operator + that adds two Vector objects and returns a new Vector object.
- An overloaded operator - that subtracts two Vector objects and returns a new Vector object.

## SOURCE CODE:

```
#include<iostream>
using namespace std;
```

```
class Vector
{
private:
```

```
int x;  
int y;
```

```
public:
```

```
    Vector(int a, int b)  
    {  
        x = a;  
        y = b;  
    }
```

```
    Vector operator + (Vector v)  
    {  
        return Vector(x + v.x, y + v.y);  
    }
```

```
    Vector operator - (Vector v)  
    {  
        return Vector(x - v.x, y - v.y);  
    }
```

```
    void display()  
    {  
        cout<<"("<<x<<","<<y<<)"<<endl;  
    }
```

```
};
```

```
int main()
```

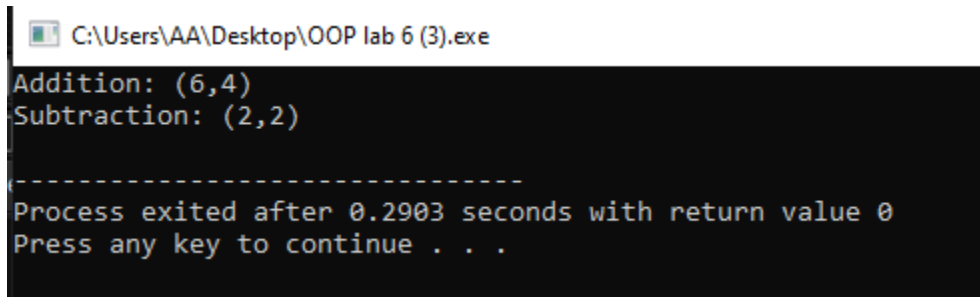
```
{  
    Vector v1(4,3);  
    Vector v2(2,1);
```

```
    Vector v3 = v1 + v2;  
    Vector v4 = v1 - v2;
```

```
    cout<<"Addition: ";
```

```
v3.display();  
  
cout<<"Subtraction: ";  
v4.display();  
  
return 0;  
}
```

## **OUTPUT:**



```
C:\Users\AA\Desktop\OOP lab 6 (3).exe  
Addition: (6,4)  
Subtraction: (2,2)  
-----  
Process exited after 0.2903 seconds with return value 0  
Press any key to continue . . .
```

# **SOLVED EXAMPLES:**

## **EXAMPLE NO: 1**

(Function Overloading Example (Compile Time Polymorphism))

### **SOURCE CODE:**

```
#include<iostream>
using namespace std;

class Overload
{
public:
    void func(int x)
    {
        cout<<"value of x is "<<x<<endl;
    }

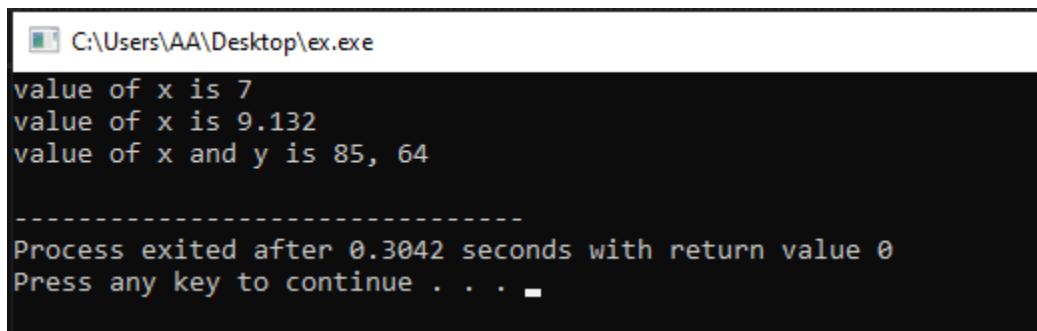
    void func(double x)
    {
        cout<<"value of x is "<<x<<endl;
    }

    void func(int x, int y)
    {
        cout<<"value of x and y is "<<x<<" "<<y<<endl;
    }
};

int main()
{
    Overload obj1;
```

```
    obj1.func(7);  
    obj1.func(9.132);  
    obj1.func(85,64);  
  
    return 0;  
}
```

## **OUTPUT:**



```
C:\Users\AA\Desktop\ex.exe  
value of x is 7  
value of x is 9.132  
value of x and y is 85, 64  
  
-----  
Process exited after 0.3042 seconds with return value 0  
Press any key to continue . . .
```

## **EXAMPLE NO: 2**

(Operator Overloading Example (Complex Number +))

## **SOURCE CODE:**

```
#include<iostream>  
using namespace std;  
  
class Complex  
{  
private:  
    float real;  
    float imag;  
  
public:
```

```

Complex()
{
    real = 0;
    imag = 0;
}

void input()
{
    cout<<"Enter real and imaginary parts respectively: ";
    cin>>real>>imag;
}

Complex operator + (const Complex& obj)
{
    Complex temp;
    temp.real = real + obj.real;
    temp.imag = imag + obj.imag;
    return temp;
}

void display()
{
    cout<<real<<" + "<<imag<<"i"<<endl;
}
};

int main()
{
    Complex c1, c2, result;

    cout<<"Enter the first complex number:"<<endl;
    c1.input();

    cout<<"Enter the second complex number:"<<endl;
    c2.input();
}

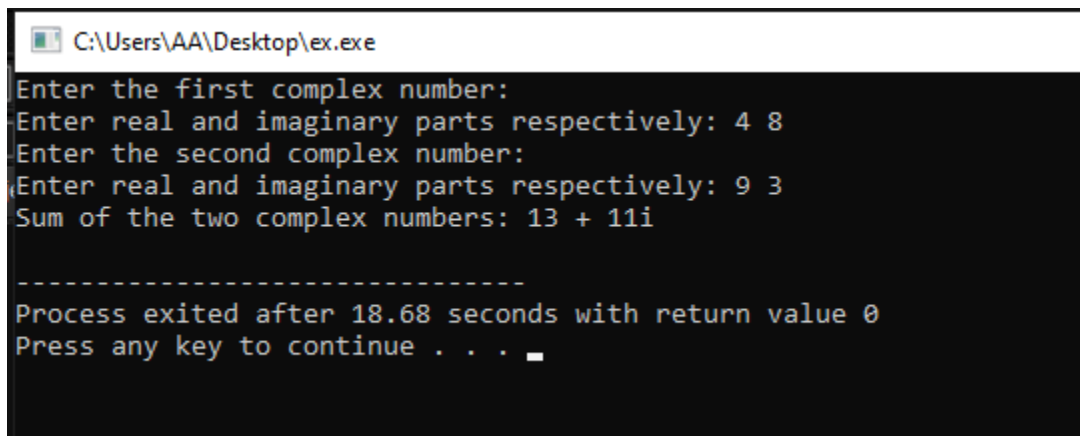
```

```
result = c1 + c2;

cout<<"Sum of the two complex numbers: ";
result.display();

return 0;
}
```

## **OUTPUT:**



```
C:\Users\AA\Desktop\ex.exe
Enter the first complex number:
Enter real and imaginary parts respectively: 4 8
Enter the second complex number:
Enter real and imaginary parts respectively: 9 3
Sum of the two complex numbers: 13 + 11i

-----
Process exited after 18.68 seconds with return value 0
Press any key to continue . . .
```

## **EXAMPLE NO: 3**

(Function Overriding (Simple Example))

## **SOURCE CODE:**

```
#include<iostream>
using namespace std;

class BaseClass
{
```

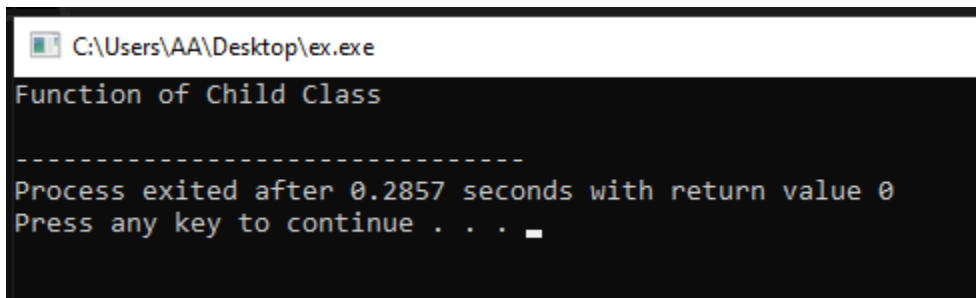
```
public:
    void disp()
    {
        cout<<"Function of Parent Class"<<endl;
    }
};

class DerivedClass : public BaseClass
{
public:
    void disp()
    {
        cout<<"Function of Child Class"<<endl;
    }
};

int main()
{
    DerivedClass obj;
    obj.disp();

    return 0;
}
```

## **OUTPUT:**



```
C:\Users\AA\Desktop\ex.exe
Function of Child Class
-----
Process exited after 0.2857 seconds with return value 0
Press any key to continue . . .
```

## **EXAMPLE NO: 4**

(Runtime Polymorphism (Virtual Function Example))

### **SOURCE CODE:**

```
#include<iostream>
using namespace std;

class BaseClass
{
public:
    virtual void display()
    {
        cout<<"This is Display() method of BaseClass"<<endl;
    }

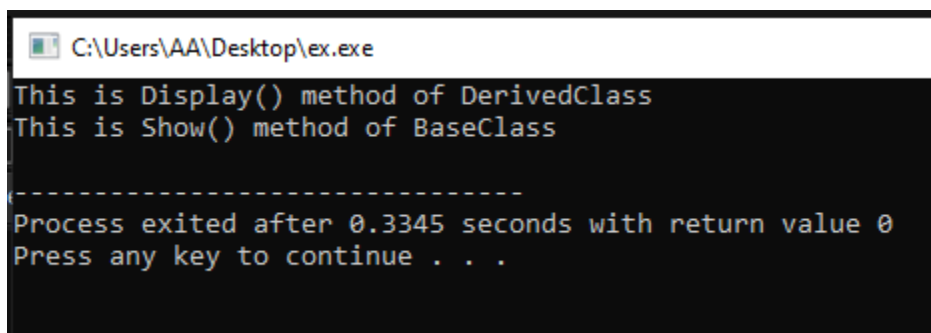
    void show()
    {
        cout<<"This is Show() method of BaseClass"<<endl;
    }
};

class DerivedClass : public BaseClass
{
public:
    void display()
    {
        cout<<"This is Display() method of DerivedClass"<<endl;
    }
};

int main()
{
```

```
BaseClass* ptr;  
DerivedClass obj;  
  
ptr = &obj;  
  
ptr->display();  
ptr->show();  
  
return 0;  
}
```

## **OUTPUT:**



```
C:\Users\AA\Desktop\ex.exe  
This is Display() method of DerivedClass  
This is Show() method of BaseClass  
-----  
Process exited after 0.3345 seconds with return value 0  
Press any key to continue . . .
```